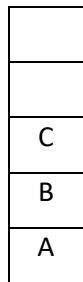


Pilha

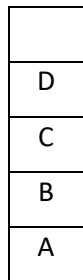
A pilha é outra estrutura fundamental na disciplina de Estruturas de Dados que se refere a uma coleção linear de elementos em que a ordem de entrada e saída dos elementos é controlada de acordo com a regra "Último a Entrar, Primeiro a Sair" (UEPS, do inglês, LIFO - *Last In, First Out*). Isso significa que o último elemento que entra na pilha será o primeiro a sair dela.

Assim, trabalha-se com uma estrutura de dados que pode ser visualizada da seguinte forma:



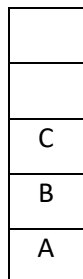
← topo

Sendo que ao realizar a inserção de um novo elemento na pilha, seguindo a regra LIFO, ele será inserido no topo da pilha, como visto abaixo, ao inserir o valor 'D':



← topo

Ainda seguindo a regra LIFO, ao remover um elemento da pilha, só o poderemos fazer no topo, removendo então o valor 'D', ficando a pilha da seguinte forma:



← topo

Assim, as principais características das pilhas são:

Inserção (Push): a inserção de elementos ocorre no topo da pilha. Quando um novo elemento é adicionado, ele se torna o novo topo da pilha.

Remoção (Pop): a remoção de elementos acontece no topo da pilha. O elemento que foi inserido mais recentemente é sempre o primeiro a ser retirado.

Restrição de acesso: os elementos da pilha não podem ser acessados diretamente, apenas por meio das operações de inserção, remoção e acesso ao topo. Isso torna a pilha uma estrutura de dados restritiva em comparação com outras, como listas e *arrays*, com acesso direto unicamente ao elemento do topo.

As pilhas podem ser implementadas de várias maneiras, como por meio de *arrays*, listas encadeadas, ou até mesmo usando duas filas. Algumas variações de pilhas são:

Pilha estática: utiliza um *array* de tamanho fixo, o que limita a quantidade máxima de elementos na pilha.

Pilha dinâmica: usa uma lista encadeada ou um *array* dinâmico para armazenar os elementos, permitindo que a pilha cresça e diminua de tamanho conforme necessário.

As pilhas são amplamente utilizadas em diversos contextos, como em sistemas operacionais para gerenciar a memória e a execução de funções, em algoritmos de simulação de eventos discretos, e em navegadores para armazenar o histórico de páginas visitadas. Além disso, são fundamentais para abordar problemas envolvendo grafos e árvores, como em buscas em profundidade (DFS - *Depth-First Search*) e para converter expressões matemáticas de notação infixada para notação posfixada (polonesa reversa).

Pilha Estática

A pilha estática é uma das maneiras de implementar a estrutura de dados conhecida como pilha, que, como já citado, segue a regra "Last In, First Out" (LIFO) – "Último a Entrar, Primeiro a Sair" (UEPS). A pilha estática utiliza um *array* de tamanho fixo para armazenar seus elementos. Devido à limitação de tamanho, é importante gerenciar o espaço do *array* de maneira eficiente para evitar a falta de espaço quando houver necessidade de armazenar mais elementos.

As funções básicas de uma pilha são:

- **Empilhar (push):** adiciona um elemento ao topo da pilha. Na implementação de pilha estática, isso significa inserir o elemento na próxima posição disponível do *array*.

- **Desempilhar (*pop*):** remove o elemento no topo da pilha. Na implementação de pilha estática, isso envolve reduzir o índice do topo em uma unidade.
- **Ver o topo:** retorna o valor do elemento no topo da pilha.
- **Verificar se a pilha está vazia:** retorna verdadeiro se a pilha estiver vazia, ou seja, se não houver elementos na pilha.
- **Verificar se a pilha está cheia:** retorna verdadeiro se a pilha estiver cheia, ou seja, se não houver espaço disponível para armazenar mais elementos.

A implementação de uma pilha estática é semelhante à implementação de uma lista estática sequencial, com a diferença de que a lista estática não segue a regra LIFO e permite o acesso direto aos elementos. Ambas as implementações utilizam arrays de tamanho fixo, mas as operações de inserção e remoção são diferentes. Na lista estática, os elementos podem ser inseridos e removidos em qualquer posição, enquanto na pilha estática, a inserção e a remoção ocorrem no topo da pilha.

Ao implementar uma pilha estática, nos concentraremos nas suas funções básicas, que incluem `push`, `pop`, `ver_topo`, `esta_vazia` e `esta_cheia`. Devido à sua relação próxima com a lista estática sequencial, a adaptaremos para atender aos requisitos de uma pilha, removendo funções desnecessárias e modificando os nomes das funções relacionadas: `inserir_fim` se torna `push` e `remover_fim` se transforma em `pop`.

Você pode observar que, embora as operações de inserção e remoção em uma pilha (`push` e `pop`) ocorram na mesma extremidade (o topo), é mais vantajoso, no caso da pilha estática, basear-se na inserção e remoção da extremidade final da lista estática sequencial e não na extremidade inicial. Vamos analisar as razões para isso:

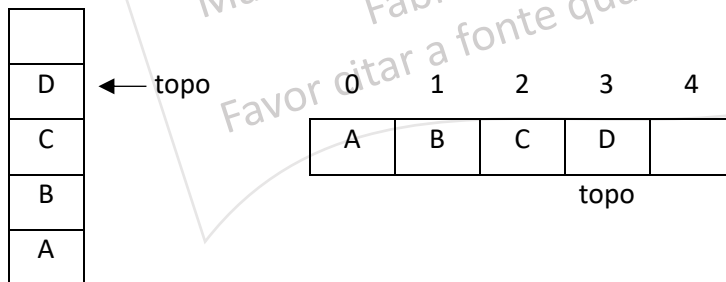
Menos movimentação de elementos: Ao realizar inserções e remoções na extremidade final da lista estática sequencial, evitamos a movimentação de todos os outros elementos da lista a cada operação. Se utilizássemos a extremidade inicial, seria necessário deslocar todos os elementos subsequentes sempre que um elemento fosse inserido ou removido, o que aumentaria a complexidade computacional e tornaria as operações mais lentas.

Acesso direto aos elementos: Como a pilha estática utiliza um *array* para armazenar os elementos, temos acesso direto a eles por meio do índice. Quando inserimos e removemos elementos na extremidade final, podemos simplesmente ajustar o índice do último elemento (topo da pilha) e acessar diretamente o elemento desejado. Se usássemos a extremidade inicial,

seria necessário atualizar todos os índices dos elementos após cada operação, tornando o processo menos eficiente.

Simplicidade na implementação: Ao utilizar a extremidade final para inserções e remoções, a implementação da pilha estática se torna mais simples, já que basta ajustar o índice do topo da pilha para realizar as operações de `push` e `pop`. Se utilizássemos a extremidade inicial, seria necessário manipular todos os índices dos elementos a cada operação, tornando a implementação mais complexa e aumentando as chances de erros.

Desta forma você pode imaginar a pilha implementada como a lista estática sequencial, “tombando-a” para a direita.



As funções básicas implementadas para a pilha estática serão, então:

- `push`: adiciona um elemento ao topo da pilha. Na implementação de pilha estática, corresponde à função inserir da lista estática sequencial, que insere o elemento na próxima posição disponível do `array`.
- `pop`: remove o elemento no topo da pilha. Na implementação de pilha estática, equivale à função remover da lista estática sequencial, que remove o elemento do topo do `array` e reduz o índice do topo em uma unidade.
- `ver_topo`: retorna o elemento no topo da pilha sem removê-lo, permitindo uma visualização do elemento.

Desta forma temos a seguinte implementação para uma pilha estática:

```
class pilha_estatica:
    def __init__(self, tamanho):
        self.tam_maximo = tamanho
        self.vetor = [None] * tamanho
        self.topo = -1
```

```

def push(self, valor):
    self.topo += 1
    self.vetor[self.topo] = valor

def pop(self):
    if not self.esta_vazia():
        self.topo -= 1

def esta_vazia(self):
    return self.topo == -1

def esta_cheia(self):
    return self.topo == self.tam_maximo - 1

def ver_topo(self):
    return self.vetor[self.topo]

def show(self):
    for i in range(self.topo + 1):
        print(self.vetor[i], end=' ')
    print()

def tamanho_atual(self):
    return self.topo + 1

def capacidade(self):
    return self.tam_maximo

```

Você notará que nesta adequação da implementação da lista estática sequencial para tornar-se uma pilha estática, o atributo `topo`, na pilha, não assumiu o propósito original do atributo `quant`, da lista estática. No caso da pilha, a inserção (`push`) e a remoção (`pop`) de elementos acontecem no topo da pilha. O `topo` é a posição do último elemento inserido na pilha e é **a partir dele** onde ocorrerão as próximas operações. Na implementação da pilha_estatica, o `topo` é determinado pelo índice do `array` que armazena os elementos. Assim, o topo da pilha é atualizado a cada inserção ou remoção, de forma que sempre aponte para a posição a partir da qual ocorrerá a próxima operação. No caso da fila, a inserção acontece no final da fila, enquanto

a remoção ocorre no início. O atributo `quant` é usado para armazenar a quantidade de elementos na fila e também serve como indicador da próxima posição disponível para inserção no array. Portanto, a cada inserção, `quant` é incrementado, apontando para a próxima posição vazia no `array`.

As funções `show`, `tamanho_atual` e `capacidade` foram mantidas na classe `pilha_estatica` porque, embora não sejam operações essenciais para o funcionamento de uma pilha, elas fornecem informações úteis sobre a estrutura e ajudam na depuração e na visualização dos elementos da pilha:

- `show`: permite exibir os elementos presentes na pilha na ordem em que estão armazenados. Essa função é útil para visualizar o estado atual da pilha e pode ser útil na depuração de problemas ou para verificar se as operações de empilhar e desempilhar estão funcionando corretamente.
- `tamanho_atual`: retorna a quantidade de elementos atualmente presentes na pilha. Essa informação pode ser útil em várias situações, como para verificar se a pilha está próxima de sua capacidade máxima ou se há espaço suficiente para adicionar mais elementos.
- `capacidade`: retorna o tamanho máximo do array que armazena os elementos da pilha. Essa informação é útil para entender a limitação de espaço da pilha estática e pode ser usada em decisões de redimensionamento ou escolha de uma estrutura de dados diferente, como uma pilha dinâmica, se necessário.

Pilha Dinâmica

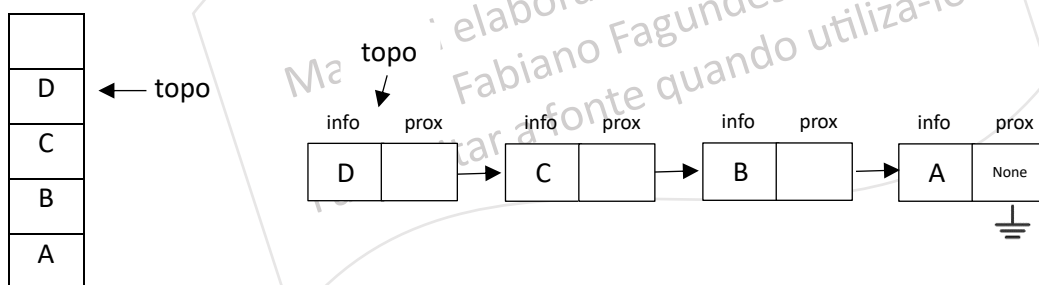
A pilha dinâmica segue o princípio *Last In, First Out* (LIFO), ou seja, o último elemento a entrar na pilha é o primeiro a sair. Diferentemente da pilha estática, que usa um `array` de tamanho fixo para armazenar os elementos, a pilha dinâmica utiliza a ideia de lista encadeada para armazenar os elementos. Neste caso, a pilha dinâmica possui um ponteiro principal apontando para o `topo` da pilha (onde ocorrem as operações de `push` e `pop`).

A implementação da pilha dinâmica oferece algumas vantagens em relação à pilha estática. Primeiramente, a pilha dinâmica pode crescer e diminuir de tamanho conforme necessário, adaptando-se às necessidades do programa em tempo de execução. Isso elimina a necessidade de definir um tamanho máximo de antemão e evita o desperdício de memória.

Na implementação da pilha dinâmica podemos observar a sua proximidade com a lista dinâmica simplesmente encadeada. Ambas são estruturas de dados baseadas em nós encadeados, sendo que a principal diferença entre elas reside nas operações permitidas e nas regras de acesso aos elementos. Assim, a pilha dinâmica pode ser implementada usando uma lista dinâmica simplesmente encadeada como base. Nesse caso, a pilha dinâmica aproveita a flexibilidade da lista dinâmica simplesmente encadeada para armazenar seus elementos e gerenciar a alocação dinâmica de memória. Porém, a pilha dinâmica restringe as operações permitidas, seguindo a regra LIFO, o que a torna mais especializada e adequada para situações específicas em que a ordem de processamento dos elementos é importante.

Embora na pilha estática utilizemos a extremidade fim da lista estática para implementar suas funções de entrada e saída (push e pop), na pilha dinâmica optamos por usar a extremidade início como referência para seu topo. Essa escolha permite que as operações de acesso, inserção e remoção sejam eficientes e simples na pilha dinâmica, uma vez que basta atualizar o ponteiro do início da lista encadeada sem a necessidade de percorrer toda a estrutura. Poderíamos ter pensado em manter o topo na extremidade fim da lista dinâmica, utilizando uma lista duplamente encadeada para reduzir o custo da remoção do elemento do fim (que seria o nosso topo). No entanto, essa abordagem não é tão eficiente quanto simplesmente inverter a extremidade da pilha dinâmica, pois exigiria o armazenamento adicional de um atributo e aumentaria a complexidade da implementação. Ao usar a extremidade início para o topo, resolvemos esse problema sem acrescentar custo algum, mantendo a implementação mais simples e eficiente.

Assim, você pode imaginar a pilha implementada como a lista dinâmica simplesmente encadeada, “tombando-a” agora para a esquerda.



Desta forma, conseguimos facilmente chegar à implementação da pilha dinâmica abaixo:

```
class No:
    def __init__(self, valor, proximo):
```

```

        self.info = valor
        self.prox = proximo

class pilha_dinamica:
    def __init__(self):
        self.topo = None
        self.quant = 0

    def push(self, valor):
        self.topo = No(valor, self.topo)
        self.quant += 1

    def pop(self):
        if not self.esta_vazia():
            self.topo = self.topo.prox
            self.quant -= 1

    def esta_vazia(self):
        return self.quant == 0

    def tamanho_atual(self):
        return self.quant

    def ver_topo(self):
        return self.topo.info

    def show(self):
        aux = self.topo
        while aux is not None:
            print(aux.info, end=' ')
            aux = aux.prox
        print()

```

Você observará que na pilha dinâmica, não há necessidade do atributo `ult` (último) porque todas as operações fundamentais de uma pilha ocorrem no topo da pilha. As operações de inserção (`push`) e remoção (`pop`) afetam apenas o topo da pilha, e não o último elemento.

No caso de uma pilha dinâmica, o atributo `topo` aponta para o primeiro elemento da lista encadeada. Quando um novo elemento é inserido (`push`), ele se torna o novo topo e passa a apontar para o elemento que era o topo anteriormente. Quando um elemento é removido (`pop`), o topo é atualizado para apontar para o próximo elemento da lista encadeada.

Dado que todas as operações relevantes ocorrem no topo da pilha, não há necessidade de rastrear o último elemento da lista encadeada. Manter o atributo `ult` só aumentaria a complexidade da implementação e consumiria memória adicional sem oferecer qualquer benefício para o funcionamento da pilha dinâmica.

As funções `show`, `tamanho_atual` e `esta_vazia` foram mantidas na classe `pilha_dinamica` porque, embora não sejam operações essenciais para o funcionamento de uma pilha, elas fornecem informações úteis sobre a estrutura e ajudam na depuração e na visualização dos elementos da pilha:

- `show`: permite exibir os elementos presentes na pilha na ordem em que estão armazenados, começando pelo `topo`. Essa função é útil para visualizar o estado atual da pilha e pode ser útil na depuração de problemas ou para verificar se as operações de `push` e `pop` estão funcionando corretamente.
- `tamanho_atual`: retorna a quantidade de elementos atualmente presentes na pilha. Essa informação pode ser útil em várias situações, como para verificar o tamanho da pilha e monitorar o uso de recursos.
- `esta_vazia`: verifica se a pilha está vazia, retornando `True` se a quantidade de elementos for zero e `False` caso contrário. Essa função é útil para evitar erros de subfluxo (*underflow*) ao tentar realizar operações de `pop` em uma pilha vazia.

Aplicações de Pilhas

As Pilhas são estruturas de dados amplamente utilizadas em diversas áreas da computação, desde a programação de baixo nível até aplicações mais complexas como sistemas de inteligência artificial. Entretanto, uma das suas aplicações mais clássicas e didáticas e que utilizaremos aqui para efeito de ilustração, é a resolução de problemas relacionados à parentização, palavras palíndromas e a utilização em cálculos no formato usado em calculadoras HP.

Na resolução do problema da parentização, a Pilha é utilizada para verificar a correspondência e o balanceamento de parênteses, colchetes e chaves em uma expressão matemática. Já para identificar palavras palíndromas, a Pilha é usada para armazenar os caracteres da palavra e depois comparar com sua versão invertida.

Além disso, a Pilha também é uma estrutura muito útil para implementar a lógica de uma calculadora HP, onde os valores e operadores são armazenados em uma Pilha para realizar as operações aritméticas.

Esses exemplos clássicos de utilização de Pilhas são importantes não apenas para entender o conceito de Pilhas, mas também para aprender a utilizar de forma eficiente e correta as suas operações básicas, como push, pop e ver_topo.

Parentização

O problema da parentização consiste em verificar se uma expressão matemática está bem parentizada, ou seja, se todos os parênteses são abertos e fechados na ordem correta. É um problema comum em programação e pode ser resolvido facilmente utilizando uma estrutura de dados chamada pilha.

Para resolver o problema, basta percorrer a expressão da esquerda para a direita e, sempre que encontrar um parêntese de abertura, empilhar na pilha. Se encontrar um parêntese de fechamento, deve-se desempilhar um parêntese da pilha. Se a pilha estiver vazia quando um parêntese de fechamento for encontrado, significa que a expressão está mal parentizada. Se, ao final do percurso da expressão, a pilha não estiver vazia, também significa que a expressão está mal parentizada.

Desta forma, utilizando o TAD Pilha implementado, não importando a estratégia utilizada, se dinâmica ou estática, temos a seguinte implementação de uma função que verifica se a parentização está correta:

```
def valida_parenteses(expressao):  
    p = pilha_dinamica.pilha_dinamica()  
    for i in range(len(expressao)):  
        if expressao[i] == '(':  
            p.push('(')  
        elif expressao[i] == ')':  
            if p.esta_vazia():  
                return False
```

```
p.pop()
return p.esta_vazia()
```

Nessa implementação, a cada abertura de parênteses encontrada, o caractere é inserido na pilha. Quando um fechamento de parênteses é encontrado, é feita a verificação se a pilha está vazia (ou seja, se não há parênteses abertos correspondentes) e, se for o caso, a expressão é inválida. Caso contrário, um parêntese é retirado da pilha para indicar que ele foi fechado corretamente.

Ao final da iteração, a função verifica se a pilha está vazia, indicando que todos os parênteses foram fechados corretamente. Se ainda houver parênteses abertos na pilha, a expressão é considerada inválida.

Por exemplo, considere a expressão $(3+2)*5-6/(2+1)$. Ao percorrer a expressão da esquerda para a direita, encontramos primeiro um parêntese de abertura e o empilhamos na pilha. Depois, encontramos outro parêntese de abertura e o empilhamos na pilha novamente. Em seguida, encontramos um parêntese de fechamento e desempilhamos um parêntese da pilha. Esse processo continua até o final da expressão. Como a pilha está vazia no final do percurso da expressão, concluímos que ela está bem parentizada. Verifique que os demais valores, símbolos etc. não são inseridos ou removidos da pilha.

Essa implementação pode ser facilmente adaptada para lidar com colchetes e chaves, utilizando o mesmo princípio. Para isso, basta adicionar mais condições no laço de verificação de caracteres. Além disso, precisamos também adicionar a lógica de que um par de colchetes ou chaves deve ser fechado antes de um par de parênteses.

```
def verifica_parenteses(expressao):
    pilha = Pilha_dinamica()

    for char in expressao:
        if char in '([{':
            pilha.push(char)
        elif char in ')]}':
            if pilha.esta_vazia():
                return False
            topo = pilha.pop()
            if (char == ')' and topo != '(') or \
```

```
(char == ']' and topo != '[') or \
(char == '}' and topo != '{'):
    return False

return pilha.esta_vazia()
```

A função `valida_parenteses` recebe como argumento uma *string* contendo a expressão matemática e utiliza um *loop* for para percorrer a *string*. Quando encontra um caractere de abertura, adiciona-o na pilha. Quando encontra um caractere de fechamento, verifica se o topo da pilha contém o caractere correspondente de abertura. Se não houver correspondência, a expressão é mal parentizada e retorna `False`. No final do *loop*, verifica se a pilha está vazia. Se estiver, a expressão é bem parentizada e retorna `True`. Se não estiver, a expressão é mal parentizada e retorna `False`.

Palavras palíndromas

O problema das palavras palíndromas consiste em verificar se uma palavra é lida da mesma forma de trás para frente, como por exemplo "arara" ou "reler". Para resolver esse problema utilizando pilhas, podemos considerar o fato de que uma pilha permite armazenar elementos em ordem inversa à sua inserção, e assim podemos utilizar essa propriedade para comparar as letras da palavra original com as letras da palavra lida ao contrário.

O algoritmo para verificar se uma palavra é palíndroma utilizando pilhas consiste em empilhar todas as letras da palavra original em uma pilha e, em seguida, desempilhar uma por uma e compará-las com as letras da palavra lida ao contrário. Se todas as letras coincidirem, a palavra é palíndroma.

Por exemplo, para a palavra "arara", podemos empilhar as letras A, R, A, R, A na pilha nessa ordem. Em seguida, desempilhamos uma por uma e comparamos com a palavra lida ao contrário, que também é "arara". Como todas as letras coincidem, concluímos que a palavra é palíndroma.

Segue abaixo um exemplo de implementação desse algoritmo em Python, utilizando a pilha dinâmica implementada anteriormente:

```
def verifica_palindromo(palavra):  
    pilha = pilha_dinamica.pilha_dinamica()  
    tam_palavra = len(palavra)  
    for i in range(tam_palavra):  
        pilha.push(palavra[i])  
    for i in range(tam_palavra):  
        if pilha.ver_topo() != palavra[i]:  
            return False  
        pilha.pop()  
    return True
```

Nesse código, a função `verifica_palindromo` recebe a palavra a ser verificada como parâmetro e utiliza a pilha dinâmica para empilhar todas as letras da palavra. Em seguida, desempilha uma por uma e compara com as letras da palavra original na ordem correta. Se todas as letras coincidirem, a função retorna `True`, indicando que a palavra é palíndroma. Caso contrário, retorna `False`.

Calculadora HP

A notação polonesa inversa, também conhecida como notação pós-fixa, é utilizada em calculadoras como a HP e em algumas linguagens de programação. Nessa notação, os operadores são colocados após os operandos, em vez de entre eles como na notação convencional.

Na lógica de entrada de dados em pilha da notação pós-fixa, cada número ou operador é empilhado, e as operações são realizadas na ordem inversa em que foram inseridas na pilha. Isso significa que, para se calcular uma expressão aritmética, por exemplo, é preciso empilhar os números e operadores na ordem pós-fixa e, em seguida, desempilhar os valores para realizar as operações.

Para implementar a lógica de entrada de dados em pilha da notação pós-fixa, é possível utilizar uma pilha. Cada número ou operador é empilhado na pilha assim que é digitado e, quando um

operador é inserido, os dois valores no topo da pilha são desempilhados para realizar a operação e o resultado é empilhado novamente na pilha.

Para implementar a lógica de entrada de dados em pilha para uma calculadora HP, podemos utilizar a nossa implementação de pilha dinâmica apresentada anteriormente.

A ideia é ler os valores e operadores da expressão na ordem posfixa e empilhar cada um na pilha. Quando um operador é lido, os dois valores mais recentes empilhados na pilha são desempilhados para realizar a operação e o resultado é empilhado novamente na pilha. Ao final da leitura da expressão, o valor resultante será o único valor presente na pilha.

Segue abaixo uma possível implementação em Python utilizando a nossa implementação de pilha dinâmica:

```
def calculadora_posfixa(expressao):
    pilha = pilha_dinamica.pilha_dinamica()

    for token in expressao.split():
        if token.isdigit():
            pilha.push(float(token))
        else:
            b = pilha.pop()
            a = pilha.pop()
            if token == '+':
                pilha.push(a + b)
            elif token == '-':
                pilha.push(a - b)
            elif token == '*':
                pilha.push(a * b)
            elif token == '/':
                pilha.push(a / b)

    return pilha.ver_topo()
```

O parâmetro da função é uma expressão matemática na notação pós-fixa, representada como uma *string*.

O primeiro passo é criar uma instância da pilha dinâmica. Em seguida, a expressão é percorrida, dividida em *tokens* (números ou operadores) utilizando o método `split()` da *string*.

Para cada token, verifica-se se é um número (`isdigit()`). Se for, é convertido para `float` e inserido na pilha utilizando o método `push`. Caso contrário, o token é considerado um operador e são retirados os dois últimos valores da pilha (`pop`), representando os operandos da operação. A operação é realizada e o resultado é inserido na pilha novamente.

Ao final da iteração, a pilha conterá apenas um valor, que é o resultado final da expressão. Esse valor é retornado utilizando o método `ver_topo` da pilha.

O código implementa a lógica de entrada de dados em pilha da calculadora HP, utilizando a pilha dinâmica implementada anteriormente para armazenar os valores e operadores inseridos pelo usuário e realizar as operações aritméticas de acordo com a lógica de entrada de dados em pilha.

Material elaborado pelo professor
Fabiano Fagundes
Favor citar a fonte quando utilizá-lo